TaskFlow

Organizador de tarefas pessoal, acadêmico e profissional

Resumo do projeto · Maria Eduarda

1. Visão geral

O TaskFlow é uma aplicação web minimalista para organizar tarefas de três áreas da vida: pessoal, estudos e trabalho. As tarefas ficam salvas na nuvem e podem ser acessadas de qualquer dispositivo (computador ou celular), com login por e-mail e senha.

- Objetivo: construir uma primeira versão funcional, simples e segura, como projeto de iniciante em programação.
- Aplicação publicada: https://taskflow-happstm94ealfcevopxbquw.streamlit.app
- Código-fonte: https://github.com/mariaeduarda98f/taskflow

2. Tecnologias utilizadas

Tecnologia	Para que foi usada
Python	Linguagem principal do projeto
Streamlit	Interface web (telas, formulários, menu lateral, tema)
Supabase (PostgreSQL)	Banco de dados na nuvem e autenticação de usuários
Row Level Security (RLS)	Regras do banco que isolam as tarefas de cada usuário
Altair + pandas	Gráficos do Painel
pytest	Testes automáticos
Git	Controle de versão: histórico de todas as mudanças do código
GitHub	Repositório na nuvem e Pull Requests
Streamlit Community Cloud	Publicação gratuita da aplicação na internet

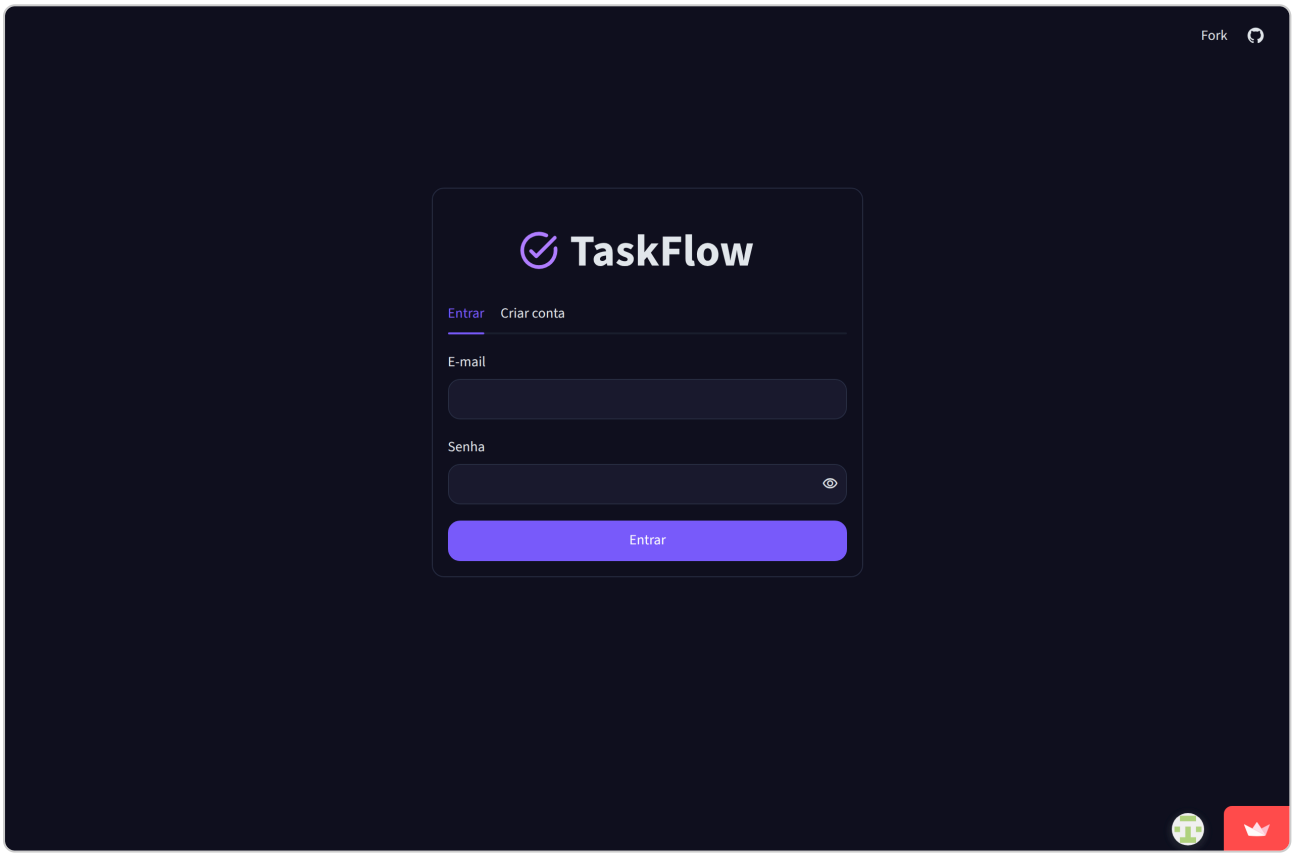
3. Funcionalidades

- Cadastro e login com e-mail e senha (Supabase Auth).
- Criar, editar, concluir e excluir tarefas.
- Cada tarefa tem: título, descrição, categoria, prioridade, status e data de entrega.

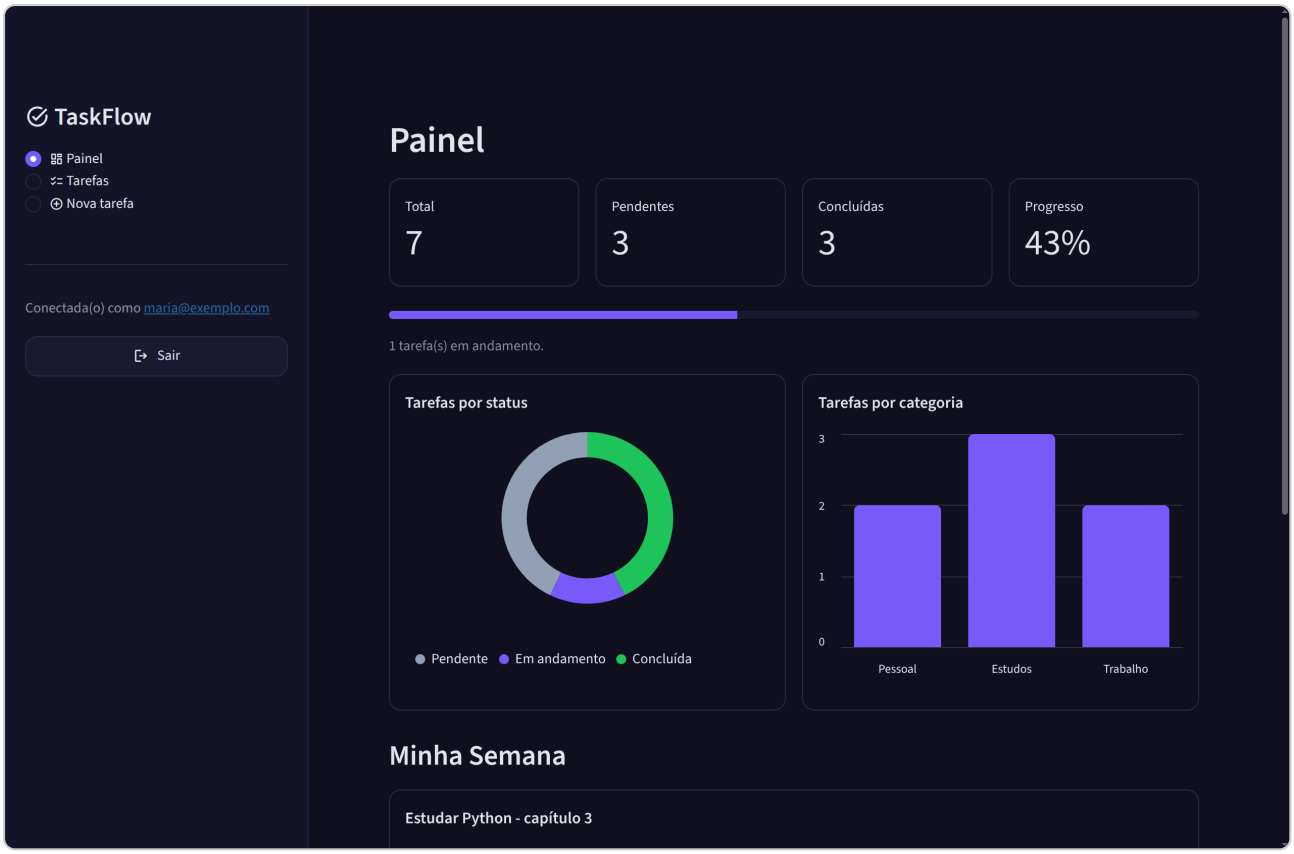
- Categorias: Pessoal, Estudos e Trabalho. Prioridades: Alta, Média e Baixa. Status: Pendente, Em andamento e Concluída.
- Filtros por categoria, prioridade e status.
- Painel com indicadores: total de tarefas, pendentes, concluídas e percentual de progresso.
- Gráficos: rosca de tarefas por status e barras de tarefas por categoria.
- Minha Semana: tarefas com prazo nos próximos 7 dias e tarefas atrasadas.
- Mensagens de erro claras em português (ex.: senha incorreta, limite de e-mails atingido).
- Tema escuro com destaque roxo e menu lateral, inspirado em um modelo de dashboard.

4. Telas da aplicação

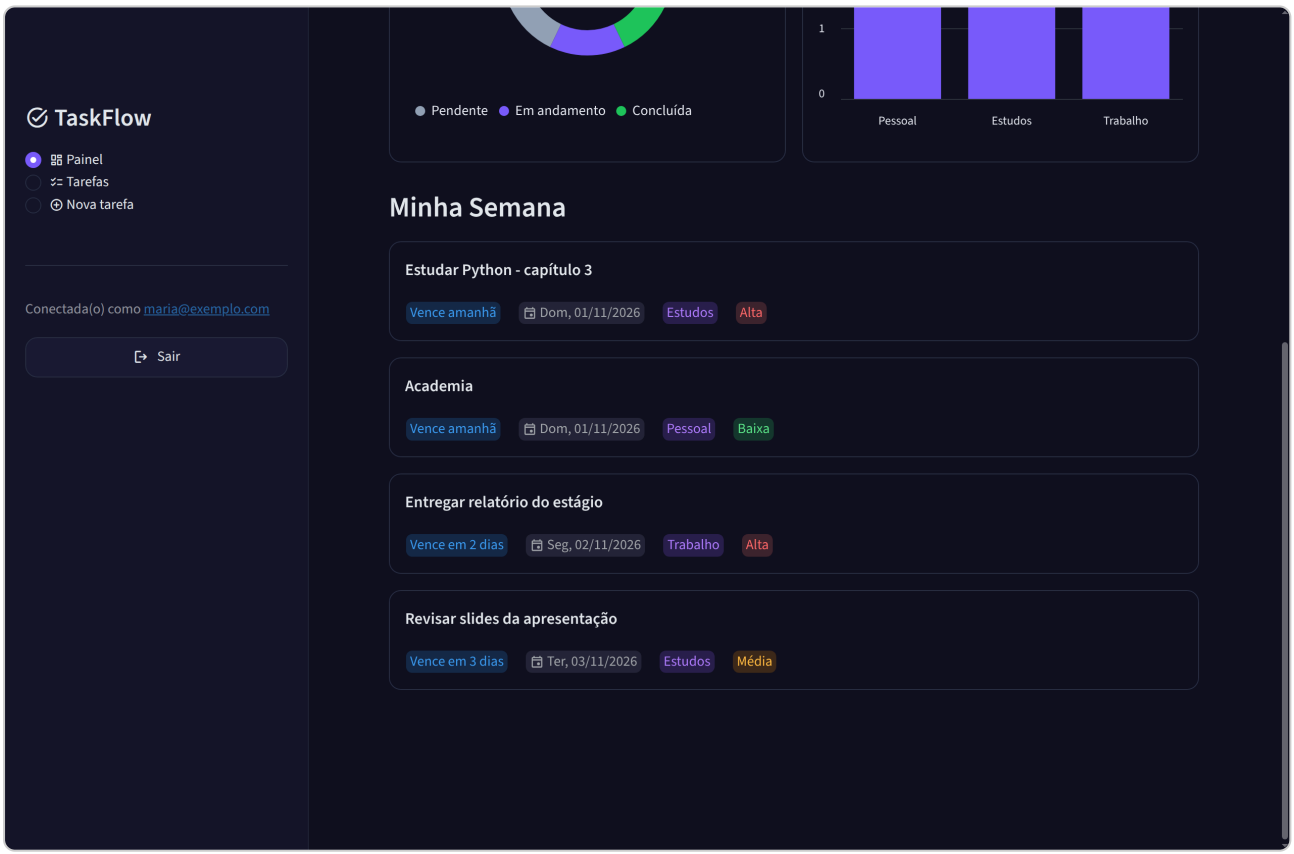
As imagens abaixo mostram as telas do TaskFlow. As telas internas usam dados de exemplo.



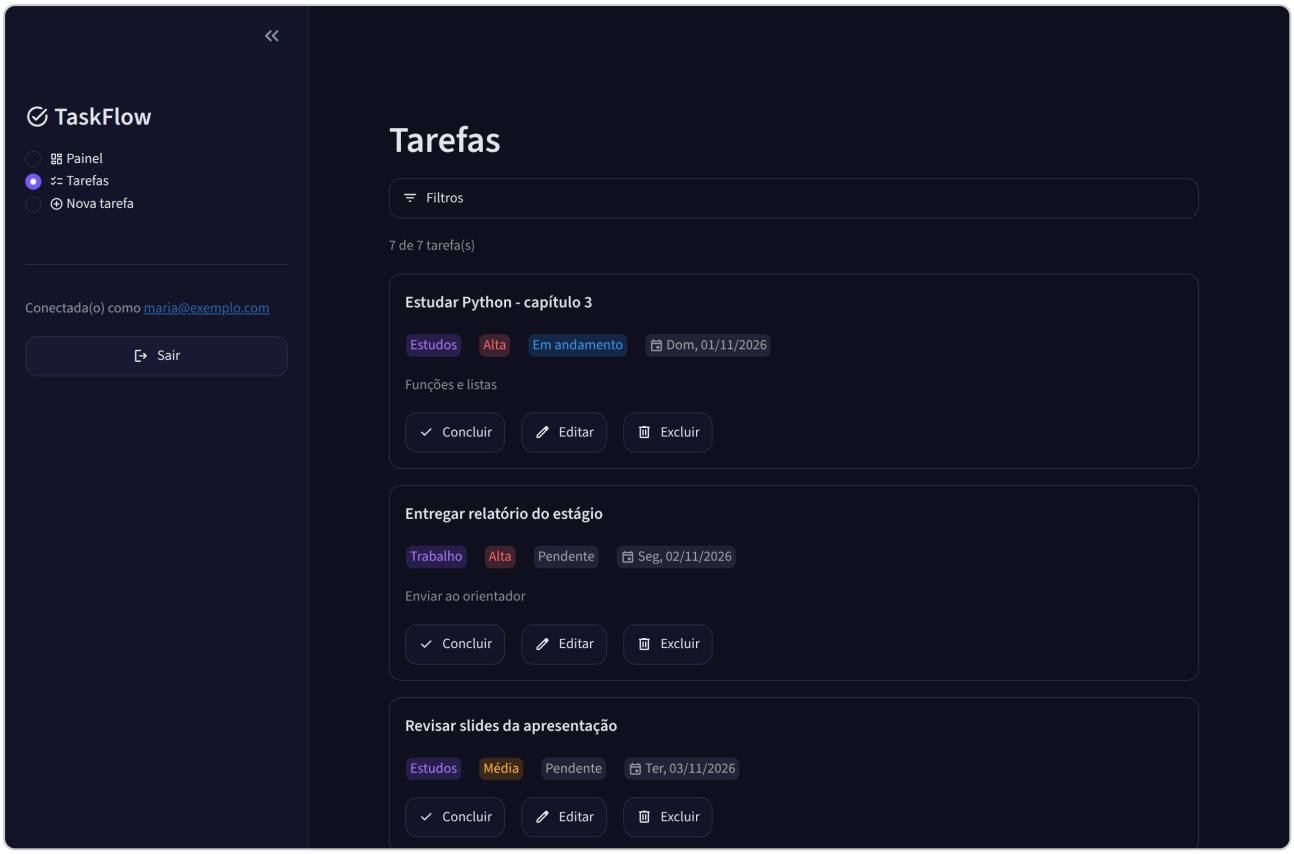
Tela de login (versão publicada)



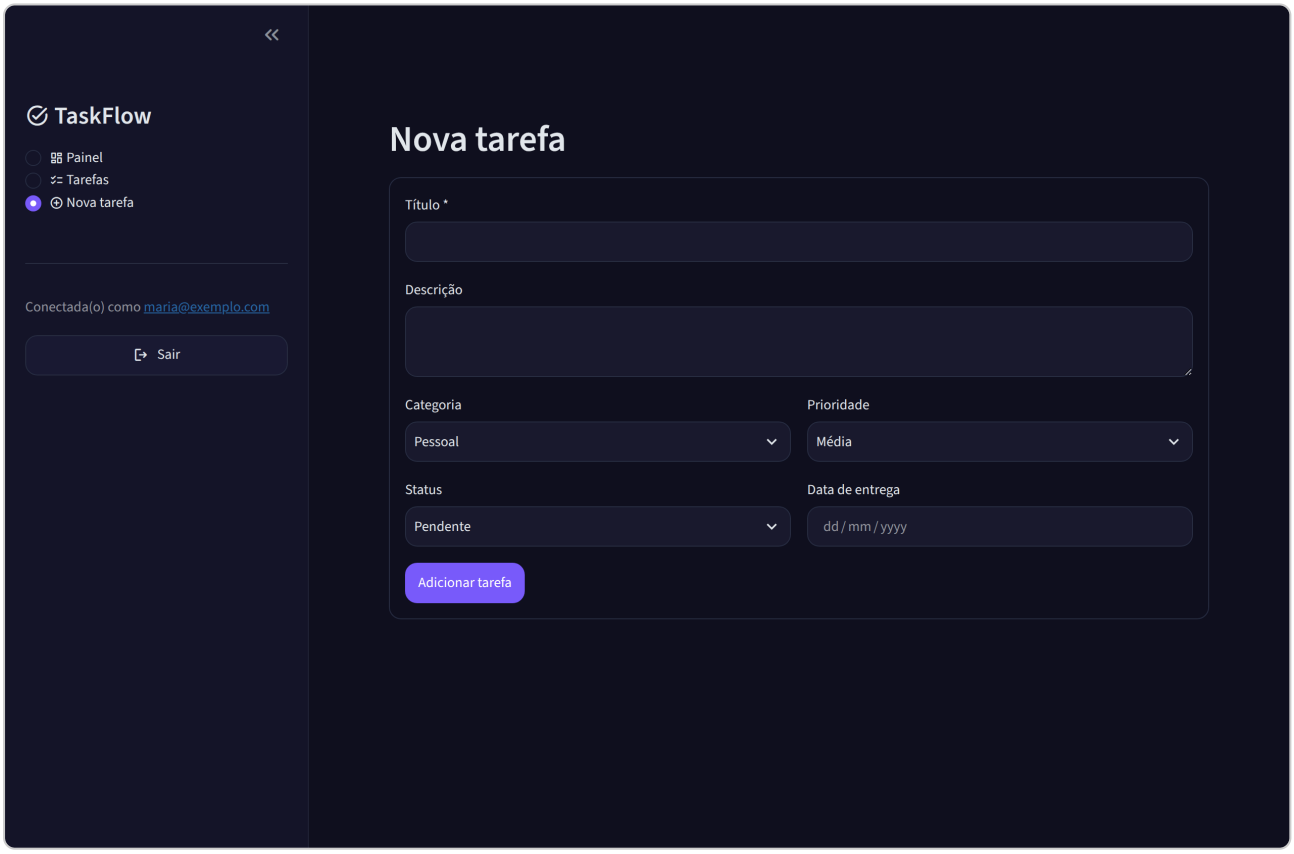
Painel: indicadores, barra de progresso e gráficos



Minha Semana: prazos dos próximos dias



Lista de tarefas com etiquetas, filtros e ações



Cadastro de nova tarefa

5. Estrutura do projeto

Arquivo / pasta	Conteúdo
app.py	Interface: login, menu lateral, Painel, Tarefas, Nova tarefa e gráficos
database.py	Conexão com o Supabase, login, operações das tarefas, filtros e indicadores
supabase/schema.sql	Criação da tabela, permissões e regras de segurança (RLS)
tests/	Testes automáticos das regras e da interface
.streamlit/config.toml	Tema visual (modo escuro e cor roxa)
requirements.txt	Bibliotecas Python usadas
.env.example / .gitignore	Modelo de configuração e proteção para segredos não irem ao GitHub
README.md	Documentação de instalação, uso, testes e publicação

6. Banco de dados

Foi criada uma tabela chamada tarefas no Supabase (PostgreSQL), com os campos: id, user_id (dono da tarefa), titulo, descricao, categoria, prioridade, status, data_entrega, criado_em e atualizado_em. O campo atualizado_em é atualizado automaticamente a cada edição. O banco também valida os valores permitidos de categoria, prioridade e status.

7. Segurança

- Row Level Security (RLS) ativado: cada usuário só consegue ver, criar, editar e excluir as próprias tarefas (regra auth.uid() = user_id). Isso vale até para quem tentar acessar a API diretamente.
- Usuários não logados não têm nenhum acesso à tabela.
- Senhas são guardadas e criptografadas pelo Supabase Auth; nunca passam pela tabela de tarefas.
- O app usa apenas a chave pública (publishable). A chave secreta, que ignora o RLS, nunca é usada.
- Nenhuma chave ou senha está no código: a configuração fica em variáveis de ambiente (.env, local) ou nos Secrets do Streamlit Cloud.
- O histórico do GitHub foi conferido e não contém segredos. O .gitignore bloqueia .env e secrets.toml.
- Confirmação de e-mail ativada no cadastro, para evitar contas falsas.
- Os registros de erro guardam só o código do erro, nunca e-mail ou senha.

8. Testes

- Testes automáticos com pytest, usando um banco falso em memória (rodam sem internet).
- Cobrem: criar, editar, concluir e excluir tarefas, validações, filtros, indicadores, Minha Semana, gráficos e mensagens de erro do login.
- Também há testes da interface (Streamlit AppTest) simulando o uso das telas.
- Resultado final: 28 testes aprovados e 2 pulados (testes de integração com o Supabase real, que só rodam quando há um usuário de teste configurado).
- Testes manuais feitos no app: cadastro, login, criar, editar, concluir, excluir, filtros e painel.

9. Etapas do desenvolvimento

Etapa	O que foi feito
PR #1: Versão 1	App completo: login, tarefas, filtros, Painel, Minha Semana, RLS, testes e documentação
PR #2: Permissões	Permissão explícita da tabela para usuários logados e bloqueio para não logados
PR #3: Mensagens de erro	Explicação clara do motivo quando o cadastro ou o login falha
PR #4: Visual novo	Tema escuro roxo, menu lateral, gráficos e tela de login em cartão
PR #5: Documentação	Link do app publicado e este resumo no README
Publicação	Deploy no Streamlit Community Cloud, com as chaves guardadas nos Secrets

10. Controle de versão com Git e GitHub

O projeto usou Git para controle de versão e GitHub para guardar o código na nuvem. Git é o programa que registra o histórico do código; GitHub é o site onde esse histórico fica publicado.

- Commits: cada mudança foi salva como um commit, com uma mensagem explicando o que mudou.
- Branches: cada melhoria foi feita em uma branch separada, sem afetar a versão principal (main) até estar pronta.
- Pull Requests: cada branch virou um Pull Request no GitHub, onde as mudanças foram revisadas e depois juntadas (merge) na main.
- Foram 5 Pull Requests: versão 1, permissões do banco, mensagens de erro, visual novo e documentação.
- Publicação automática: o Streamlit Community Cloud lê o código da branch main; cada merge atualiza o app publicado.
- Segredos fora do Git: o arquivo .gitignore impede que .env e secrets.toml (com chaves) sejam enviados ao repositório.

Comando	Para que serve
git clone	Baixar o repositório para o computador
git checkout -b	Criar uma branch nova para uma melhoria
git add / git commit	Salvar uma mudança no histórico
git push	Enviar as mudanças para o GitHub

11. Desafios e soluções

Problema	Solução
Tabela não encontrada no Supabase	Execução do schema.sql no SQL Editor
"Permission denied" ao ler as tarefas	Concessão de permissão (grant) para usuários logados; o RLS continuou limitando cada um às próprias tarefas
Limite de envio de e-mails no cadastro	Mensagem clara no app e criação do usuário com confirmação automática pelo painel do Supabase
Mensagens de erro genéricas	Tradução dos códigos de erro do Supabase para mensagens em português
Streamlit Cloud não encontrava o repositório	O repositório era privado; foi tornado público (sem segredos no código)
Erro de formato nos Secrets	Correção do texto no formato TOML com as duas variáveis

12. Próximos passos (melhorias futuras)

- Configurar um servidor de e-mail próprio (SMTP) para outras pessoas criarem conta sem limite de e-mails.
- Aumentar o tamanho mínimo de senha para 8 caracteres.
- Ajuste de segurança recomendado pelo Supabase na função de data de modificação (search_path).
- Novas funções possíveis: lembretes, subtarefas, quadro Kanban e tema claro/escuro selecionável.